TP 1 : construire et optimiser un data warehouse ShopFlow

Travail en binôme.
Plateformes : **BigQuery** (sandbox, sans carte bancaire) et **Databricks Free Edition**.
Prérequis : avoir suivi le cours de la séance 1 et terminé la préparation ci-dessous **avant** le TP.

> Contexte : vous êtes data engineers chez ShopFlow, un site e-commerce européen. L'équipe fraude veut un entrepôt de données pour suivre les ventes et les paiements frauduleux, et la direction veut savoir combien coûtent les requêtes. Vous allez construire le même entrepôt sur deux plateformes, puis mesurer l'effet de l'organisation physique des tables sur la quantité de données lues.

Préparation (à faire avant la séance)

1. **BigQuery sandbox** : ouvrez la console BigQuery avec un compte Google, créez un projet (ex. `shopflow-<nom>`). Aucune carte bancaire n'est demandée. Vérifiez que le bandeau « Sandbox » est affiché.
2. **Databricks Free Edition** : créez un compte sur databricks.com/learn/free-edition et vérifiez que vous accédez à l'éditeur SQL et aux notebooks.
3. Récupérez le fichier `shopflow_gen.py` fourni par l'enseignant.

Limites à connaître :

Préparation (à faire avant la séance) (suite)

- Le sandbox BigQuery n'autorise **ni** `INSERT`, **ni** `UPDATE`, **ni** `DELETE`, **ni** `MERGE` : toutes les tables se créent avec `CREATE TABLE ... AS SELECT`. Les tables expirent au bout de 60 jours. Le stockage est limité à 10 Gio **au total sur la vie du sandbox** : ne multipliez pas les copies inutiles.
- Databricks Free Edition ne fournit qu'un SQL warehouse de taille 2X-Small et peut couper le calcul pour la journée en cas d'abus.

Partie 0 : modélisation sur papier

Voici les données sources de ShopFlow (générées, 24 mois : 2024 et 2025) :

Table	Grain	Lignes (SF1)	Colonnes principales
`customers`	un client	50 001	`customer_id`, `country`, `signup_date`, `segment` (le client `0` représente les commandes invité)
`products`	un produit	2 000	`product_id`, `category`, `brand`, `price`
`orders`	une commande	1 011 946	`order_id`, `customer_id`, `order_ts`, `status`, `shipping_country`, `total_amount`, `device`, `ip`
`order_items`	une ligne de commande	1 818 927	`order_id`, `line_no`, `product_id`, `quantity`, `unit_price`
`payments`	une tentative de paiement	1 031 980	`payment_id`, `order_id`, `customer_id`, `card_id`, `payment_ts`, `amount`, `payment_method`, `card_country`, `status`, `is_fraud`, `fraud_scenario`, `chargeback_ts`
`clickstream`	un événement de navigation	15 984 193	`event_id`, `session_id`, `customer_id`, `event_ts`, `event_type`, `page`, `product_id`, `device`, `ip`

Répondez par écrit, en binôme :

Partie 0 : modélisation sur papier (suite)

1. Quels sont les **processus métier** à modéliser ? Pour chacun, quel est le **grain** de la table de faits ?
2. Quelles **dimensions** partagent ces tables de faits ?
3. Quelles colonnes sont des **mesures** additives ? Lesquelles ne le sont pas ?
4. `payments.is_fraud` est-il connu au moment du paiement ? Quelle colonne représente ce que l'entreprise apprend réellement, et quand ?
5. Comment représenter dans `dim_customer` les commandes passées sans compte (client `0`) ?

Partie 1 : accès aux données

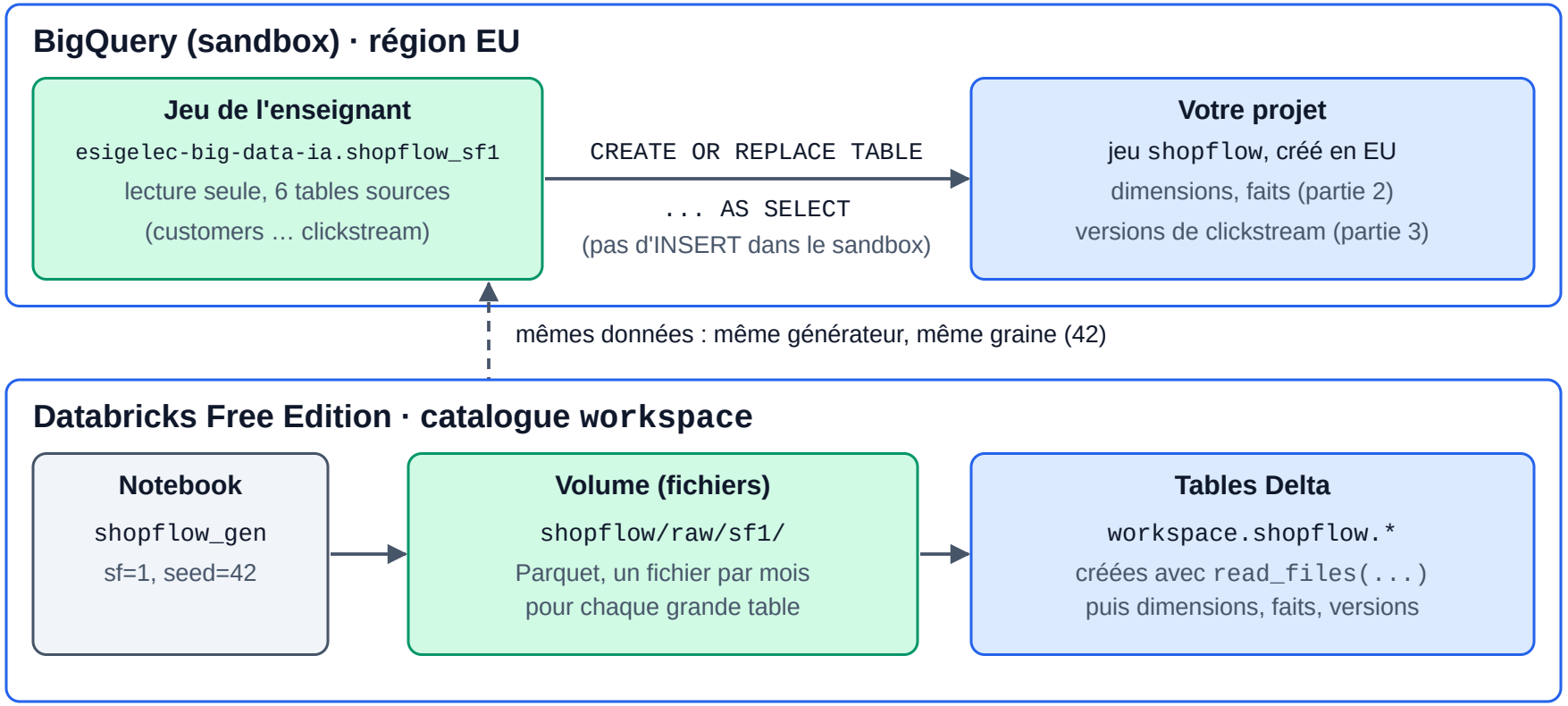


Schéma : dans BigQuery, vous lisez le jeu de l'enseignant et écrivez dans le vôtre (tous deux en EU) ; dans Databricks, vous générez vous-mêmes les fichiers, identiques grâce à la même graine.

1.A BigQuery : jeu de données partagé par l'enseignant

L'enseignant a chargé les données dans un jeu de données en lecture seule : `esigelec-big-data-ia.shopflow_sf1`, situé dans la région **EU**.

1. Dans votre projet, créez votre propre jeu de données `shopflow`, **dans la région EU** (obligatoire : une requête ne peut pas lire et écrire dans deux régions différentes) :

```
CREATE SCHEMA IF NOT EXISTS shopflow OPTIONS (location = 'EU');
```

1.A BigQuery : jeu de données partagé par l'enseignant (suite)

2. **Désactivez le cache** pour toute la séance : dans l'éditeur, *Plus (ou Edit) > Paramètres de requête (Query settings) > Préférence de cache : décocher « Utiliser les résultats mis en cache » (Use cached results)*. Sinon, une requête relancée affiche 0 octet traité et vos mesures sont faussées.
3. Explorez une table **sans lancer de requête** : ouvrez `payments` dans l'explorateur, onglets *Schéma*, *Détails* (taille, nombre de lignes) et *Aperçu*. L'aperçu est gratuit, contrairement à un `SELECT * ... LIMIT 10`.
4. **Point de contrôle** : comptez les lignes de chaque table et comparez au tableau de la partie 0.

```sql
SELECT 'payments' AS t, COUNT(*) AS n FROM `esigelec-big-data-ia.shopflow_sf1.payments`
UNION ALL SELECT 'orders', COUNT(*) FROM `esigelec-big-data-ia.shopflow_sf1.orders`;
-- complétez pour les 6 tables
```

Notez les octets traités : pourquoi un `COUNT(*)` coûte-t-il si peu ?

1.A BigQuery : jeu de données partagé par l'enseignant (suite)

5. Créez **vos propres tables** dans `shopflow` à partir des tables de l'enseignant, avec `CREATE TABLE ... AS SELECT` (la table est créée et remplie par le résultat du `SELECT`, en une seule commande) :

```
CREATE OR REPLACE TABLE shopflow.customers   AS SELECT * FROM `esigelec-big-data-ia.shopflow_sf1.customers`;
CREATE OR REPLACE TABLE shopflow.products    AS SELECT * FROM `esigelec-big-data-ia.shopflow_sf1.products`;
CREATE OR REPLACE TABLE shopflow.orders      AS SELECT * FROM `esigelec-big-data-ia.shopflow_sf1.orders`;
CREATE OR REPLACE TABLE shopflow.order_items AS SELECT * FROM `esigelec-big-data-ia.shopflow_sf1.order_items`;
CREATE OR REPLACE TABLE shopflow.payments    AS SELECT * FROM `esigelec-big-data-ia.shopflow_sf1.payments`;
CREATE OR REPLACE TABLE shopflow.clickstream AS SELECT * FROM `esigelec-big-data-ia.shopflow_sf1.clickstream`;
```

Vérifiez dans l'explorateur que les 6 tables apparaissent sous `shopflow`, avec les mêmes nombres de lignes. Notez les octets traités : contrairement au `COUNT(*)`, une copie lit toutes les colonnes.

1.B Databricks : générer les mêmes données

Databricks Free Edition n'a pas accès à Internet librement : on génère donc les données directement dans la plateforme avec le même générateur et la même graine. Les données seront identiques à celles de BigQuery.

1. Dans l'éditeur SQL, créez un schéma et un volume (espace de fichiers géré par Unity Catalog) :

```
CREATE SCHEMA IF NOT EXISTS workspace.shopflow;
CREATE VOLUME IF NOT EXISTS workspace.shopflow.raw;
```

1.B Databricks : générer les mêmes données (suite)

2. Dans *Workspace*, créez un dossier `tp1`, importez-y `shopflow_gen.py`, puis créez dans ce même dossier un notebook Python `01_generation`.

3. Dans le notebook, exécutez (durée attendue : quelques minutes). Sur le calcul serverless, le dossier courant n'est pas garanti : on ajoute donc explicitement le chemin du dossier `tp1` (visible en haut du navigateur *Workspace*) :

```
import sys
sys.path.append("/Workspace/Users/<votre_email>/tp1")
import shopflow_gen
counts = shopflow_gen.generate(out="/Volumes/workspace/shopflow/raw/sf1", sf=1, seed=42)
counts
```

4. Vérifiez la présence des fichiers : *Catalog > workspace > shopflow > raw*. Chaque grande table est découpée en un fichier Parquet par mois.

5. Créez une table Delta par source, en lisant les fichiers Parquet :

1.B Databricks : générer les mêmes données (suite)

```
CREATE OR REPLACE TABLE workspace.shopflow.payments AS
SELECT * FROM read_files('/Volumes/workspace/shopflow/raw/sf1/payments/', format => 'parquet');
-- même chose pour customers, products, orders, order_items, clickstream
```

6. Désactivez le cache de résultats et fixez le fuseau horaire en UTC (comme BigQuery), à chaque nouvelle session SQL :

```
SET use_cached_result = false;
SET TIME ZONE 'UTC';
```

7. **Point de contrôle** : mêmes nombres de lignes que dans BigQuery.

Partie 2 : schéma en étoile et questions métier

Construisez sur **les deux plateformes** le schéma suivant. Dans BigQuery, les tables vont dans `shopflow` et lisent `esigelec-big-data-ia.shopflow_sf1` ; dans Databricks, tout est dans `workspace.shopflow`. Utilisez uniquement `CREATE OR REPLACE TABLE ... AS SELECT`.

Partie 2 : schéma en étoile et questions métier (suite)

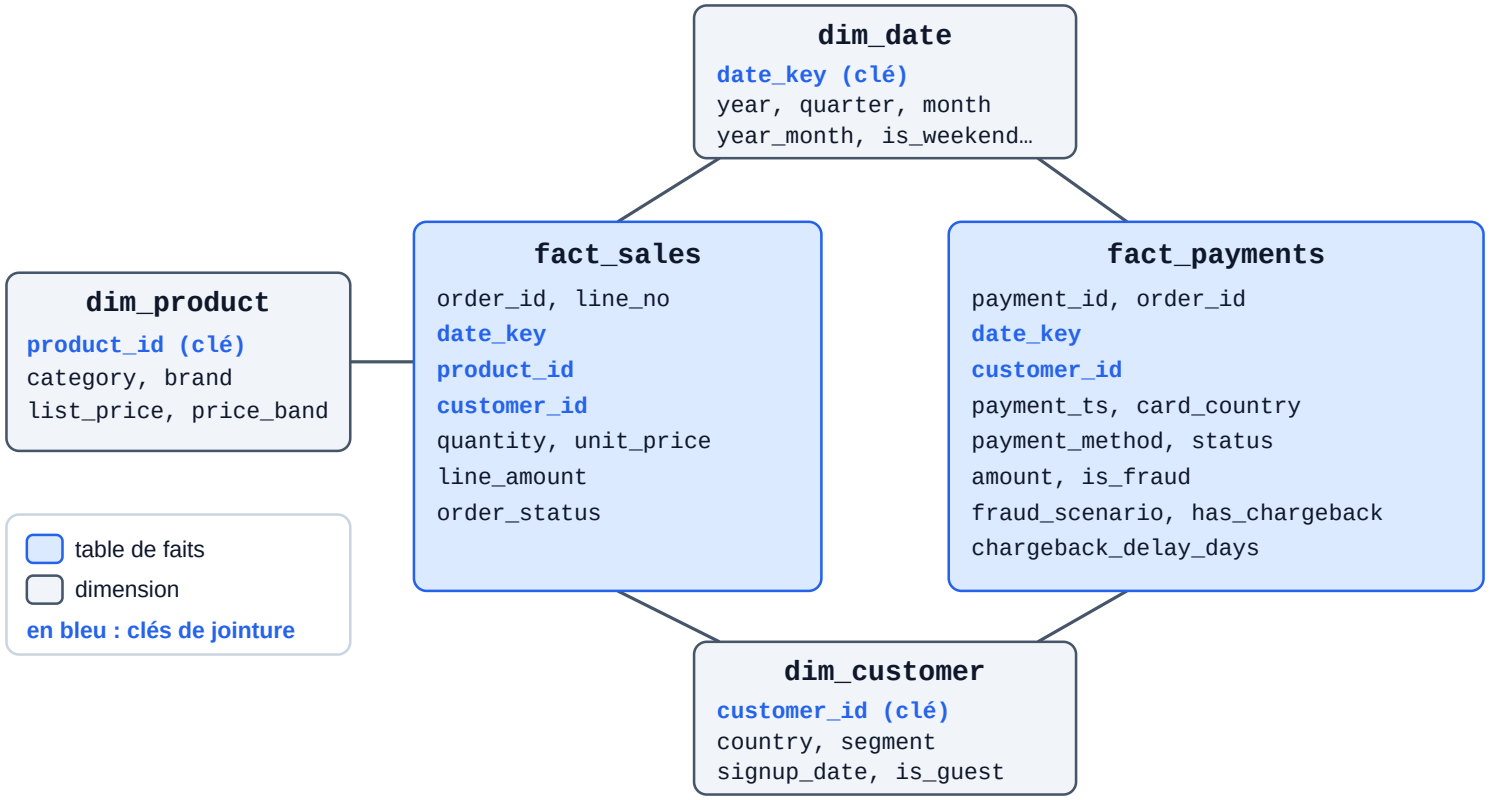


Schéma : chaque trait relie une table de faits à une dimension ; les colonnes en bleu sont les clés de jointure, le détail des colonnes est dans le tableau ci-dessous.

Partie 2 : schéma en étoile et questions métier (suite)

Table	Grain	Colonnes attendues
`dim_date`	un jour du 01/01/2024 au 31/12/2025	`date_key` (DATE), `year`, `quarter`, `month`, `year_month` (texte `AAAA-MM`), `day_of_week`, `is_weekend`
`dim_customer`	un client, y compris l'invité	`customer_id`, `country` (`'Inconnu'` si absent), `segment`, `signup_date`, `is_guest`
`dim_product`	un produit	`product_id`, `category`, `brand`, `list_price`, `price_band` (`<20`, `20-100`, `100-500`, `>500`)
`fact_sales`	une ligne de commande	`order_id`, `line_no`, `date_key`, `customer_id`, `product_id`, `quantity`, `unit_price`, `line_amount`, `order_status`
`fact_payments`	une tentative de paiement	`payment_id`, `order_id`, `date_key`, `payment_ts`, `customer_id`, `card_country`, `payment_method`, `status`, `amount`, `is_fraud`, `fraud_scenario`, `has_chargeback`, `chargeback_delay_days`

Aides :

- BigQuery : `GENERATE_DATE_ARRAY` + `UNNEST` pour `dim_date` ; `DATE(ts)`, `FORMAT_DATE`, `EXTRACT`, `TIMESTAMP_DIFF`.
- Databricks : `explode(sequence(DATE'2024-01-01', DATE'2025-12-31', INTERVAL 1 DAY))` ; `date_format`, `dayofweek`, `datediff`.

Points de contrôle :

Partie 2 : schéma en étoile et questions métier (suite)

- `dim_date` : 731 lignes.
- `dim_customer` : 50 001 lignes, dont 1 invité.
- `fact_sales` : autant de lignes que `order_items`. `fact_payments` : autant que `payments`.
- `SUM(line_amount)` par commande doit être égal à `orders.total_amount` (écrivez la requête de contrôle).

Questions métier (à écrire sur le schéma en étoile, sur les deux plateformes) :

- **B1.** Chiffre d'affaires mensuel 2025 par catégorie de produit, en excluant les commandes annulées.
- **B2.** Pour les paiements autorisés, taux de fraude par pays de carte, en nombre et en montant. Gardez les 5 pays au taux en montant le plus élevé.
- **B3.** Pour chaque scénario de fraude : nombre de paiements, montant total, part des fraudes autorisées ayant reçu une rétrofacturation, délai moyen de rétrofacturation en jours.

Partie 3 : expérience, partitionnement et clustering

On travaille sur la plus grande table, `clickstream` (environ 16 millions d'événements), sous trois organisations physiques.

3.A Préparer les trois versions

BigQuery

- **V0** : la table partagée telle quelle, `esigelec-big-data-ia.shopflow_sf1.clickstream` (ni partitionnée, ni clusterisée).
- **V1** : partitionnée par mois.

```
CREATE OR REPLACE TABLE shopflow.events_part
PARTITION BY TIMESTAMP_TRUNC(event_ts, MONTH) AS
SELECT * FROM `esigelec-big-data-ia.shopflow_sf1.clickstream`;
```

- **V2** : partitionnée par mois **et** clusterisée par `customer_id`, `event_type` (à écrire vous-mêmes).

3.A Préparer les trois versions (suite)

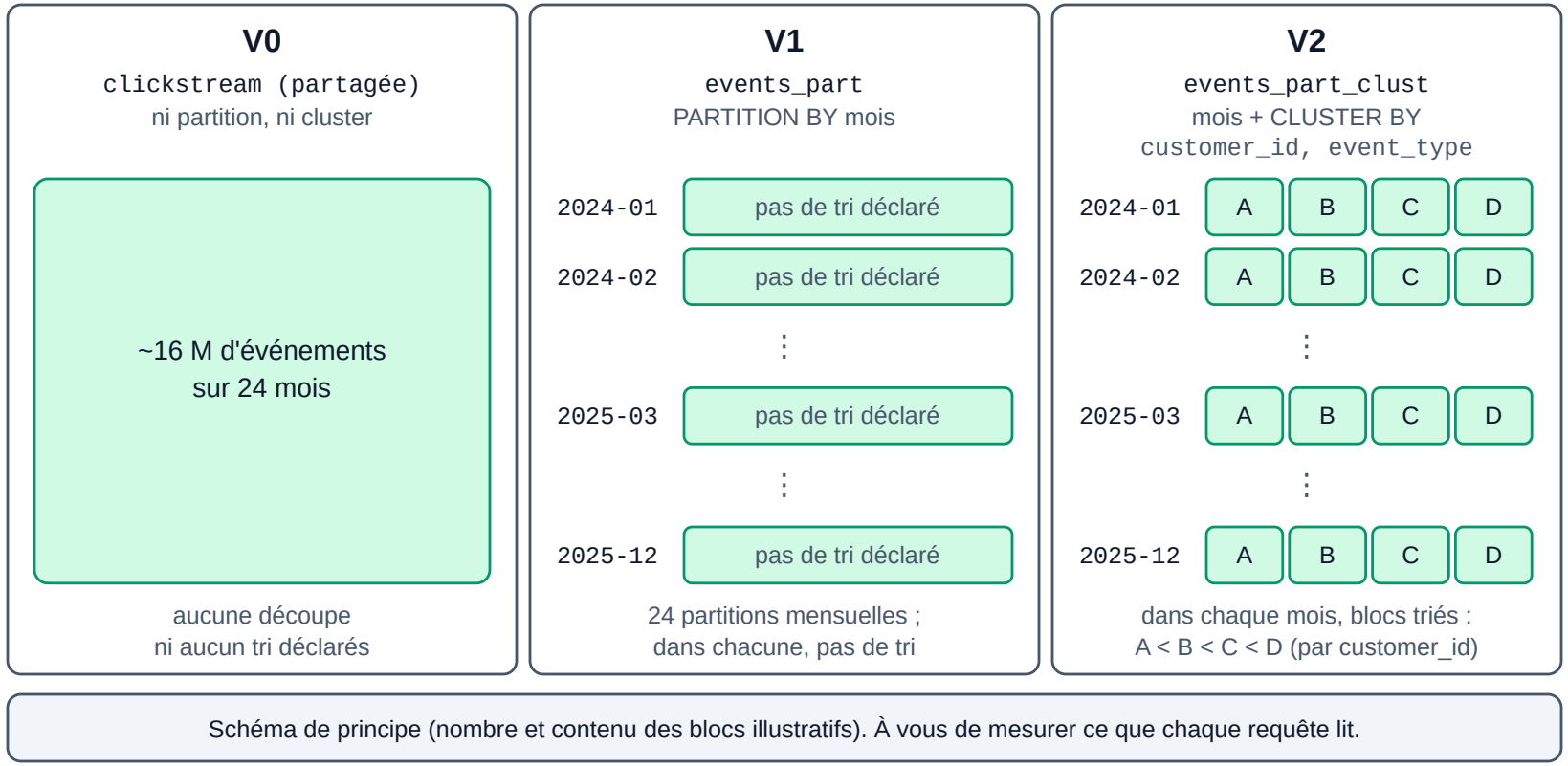


Schéma de principe des trois versions BigQuery : regardez comment chaque organisation découpe la table, puis prédisez, avant de mesurer, ce que chaque requête pourra éviter de lire.

3.A Préparer les trois versions (suite)

Databricks

- **V0** : `events_shuffled`, une copie de `clickstream` dans un ordre aléatoire (simule une table alimentée sans aucun ordre) :

```
CREATE OR REPLACE TABLE workspace.shopflow.events_shuffled AS
SELECT * FROM workspace.shopflow.clickstream ORDER BY rand();
```

- **V1** : `events_part`, partitionnée par mois, comme la V1 de BigQuery. Attention : la table `clickstream` créée en partie 1 **n'est pas partitionnée** (aucun `PARTITIONED BY`), il faut donc créer cette version. Delta ne sait pas partitionner directement sur une expression : on ajoute une **colonne générée** `event_month`, calculée automatiquement à partir de `event_ts`, et on partitionne sur elle. On crée d'abord la table vide avec son schéma, puis on la remplit :

3.A Préparer les trois versions (suite)

```
CREATE OR REPLACE TABLE workspace.shopflow.events_part (
 event_id    BIGINT,
 session_id  BIGINT,
 customer_id BIGINT,
 event_ts    TIMESTAMP,
 event_type  STRING,
 page        STRING,
 product_id  BIGINT,
 device      STRING,
 ip          STRING,
 event_month STRING GENERATED ALWAYS AS (DATE_FORMAT(event_ts, 'yyyy-MM'))
)
PARTITIONED BY (event_month);

INSERT INTO workspace.shopflow.events_part
 (event_id, session_id, customer_id, event_ts, event_type, page, product_id, device, ip)
SELECT event_id, session_id, customer_id, event_ts, event_type, page, product_id, device, ip
FROM workspace.shopflow.clickstream;

DESCRIBE DETAIL workspace.shopflow.events_part;  -- partitionColumns doit afficher ["event_month"]
```

3.A Préparer les trois versions (suite)

Si l'`INSERT` refuse un type, comparez avec `DESCRIBE TABLE workspace.shopflow.clickstream` et ajustez le schéma.

- **V2** : `events_lc`, avec *liquid clustering* sur `customer_id` et `event_ts`, puis `OPTIMIZE` (à écrire vous-mêmes).

Pour chaque version, relevez le nombre de fichiers et la taille avec `DESCRIBE DETAIL <table>`.

3.B Les cinq requêtes

Écrivez-les une fois, puis exécutez-les sur V0, V1 et V2 en changeant seulement le nom de la table (`T`).

```sql
-- Q1 : volume par type d'événement en mars 2025
SELECT event_type, COUNT(*) AS n
FROM T
WHERE event_ts >= TIMESTAMP '2025-03-01' AND event_ts < TIMESTAMP '2025-04-01'
GROUP BY event_type;

-- Q2 : parcours complet du client 4242
SELECT event_ts, session_id, event_type, page, product_id
FROM T
WHERE customer_id = 4242
ORDER BY event_ts;

-- Q3 : aperçu de la table
SELECT * FROM T LIMIT 10;

-- Q4 : sessions distinctes par jour en novembre 2025
SELECT DATE(event_ts) AS jour, COUNT(DISTINCT session_id) AS sessions
FROM T
WHERE event_ts >= TIMESTAMP '2025-11-01' AND event_ts < TIMESTAMP '2025-12-01'
GROUP BY jour
ORDER BY jour;
```

3.B Les cinq requêtes (suite)

```sql
-- Q5 : ajouts au panier du client 4242 en 2025
SELECT COUNT(*) AS ajouts
FROM T
WHERE customer_id = 4242 AND event_type = 'add_to_cart'
  AND event_ts >= TIMESTAMP '2025-01-01' AND event_ts < TIMESTAMP '2026-01-01';
```

3.C Mesurer

BigQuery : pour chaque requête, notez

1. l'**estimation avant exécution** affichée par l'éditeur (*dry run*, « Cette requête va traiter X ») ;
2. les **octets réellement facturés** après exécution, que vous retrouverez ensuite toutes ensemble avec :

```sql
SELECT creation_time,
       ROUND(total_bytes_processed / POW(1024, 2), 1) AS mo_traites,
       ROUND(total_bytes_billed / POW(1024, 2), 1) AS mo_factures,
       total_slot_ms, cache_hit,
       LEFT(query, 80) AS debut_requete
FROM `region-eu`.INFORMATION_SCHEMA.JOBS_BY_USER
WHERE creation_time > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR)
ORDER BY creation_time DESC;
```

3.C Mesurer (suite)

Les colonnes `total_bytes_*` sont en octets : on les divise par 1024 × 1024 pour obtenir des Mo, l'unité qu'affiche aussi l'éditeur BigQuery.

Databricks : exécutez dans l'éditeur SQL, puis ouvrez *Query History > votre requête > See query profile*. Notez la durée, les octets lus et la part de données élaguées (*pruned*).

Complétez le tableau :

Requête	BQ V0 estimé / facturé (Mo)	BQ V1 estimé / facturé (Mo)	BQ V2 estimé / facturé (Mo)	DBX V0 Mo lus / % élagué	DBX V1	DBX V2
Q1						
Q2						
Q3						
Q4						
Q5						

Questions :

3.C Mesurer (suite)

1. Sur V0, pourquoi Q3 (`LIMIT 10`) coûte-t-elle plus cher que Q2 dans BigQuery ? Le résultat est-il différent sur V2 ?
2. Sur V2, l'estimation avant exécution et les octets facturés diffèrent-ils ? Pourquoi ?
3. Cherchez dans la page de tarifs la quantité minimale facturée par table lue. Quel effet sur vos petites requêtes ?
4. Dans Databricks, vos requêtes filtrent sur `event_ts`, alors que V1 est partitionnée sur `event_month`. Comment le moteur sait-il quelles partitions ignorer ? Vérifiez-le avec `EXPLAIN` sur Q1.
5. La documentation BigQuery conseille d'éviter les partitions de moins de 10 Go environ. Nos partitions mensuelles font beaucoup moins : que recommanderiez-vous en production pour cette table ?

Partie 4 : corriger trois requêtes

Chaque requête ci-dessous est soit coûteuse, soit **fausse**. Pour chacune : exécutez-la, expliquez le problème, réécrivez-la, et mesurez le gain (BigQuery). Adaptez la syntaxe pour Databricks si besoin.

```sql
-- M1 : « je veux les événements du client 4242 »
SELECT *
FROM (SELECT * FROM shopflow.events_part_clust)
WHERE customer_id = 4242;

-- M2 : « nombre d'événements en mars 2025 »
SELECT COUNT(*)
FROM shopflow.events_part
WHERE FORMAT_TIMESTAMP('%Y-%m', event_ts) = '2025-03';

-- M3 : « montant fraudé par pays de carte »
SELECT p.card_country, SUM(p.amount) AS montant_fraude
FROM shopflow.fact_payments AS p
JOIN shopflow.fact_sales AS s ON s.order_id = p.order_id
WHERE p.is_fraud AND p.status = 'authorized'
GROUP BY p.card_country
ORDER BY montant_fraude DESC;
```

Indice pour M3 : comparez le total obtenu avec `SELECT SUM(amount) FROM shopflow.fact_payments WHERE is_fraud AND status = 'authorized'`.

Partie 5 : restitution

Chaque binôme présente brièvement :

1. La requête qui a le plus gagné entre V0 et V2, et pourquoi, en lien avec le cours (stockage colonnaire, élagage de blocs).
2. Une différence de comportement entre BigQuery et Databricks que vous avez observée.
3. Le coût estimé de Q2 sur V0 si la table faisait 10 To, au tarif *on-demand* de 6,25 $ par Tio.

Si quelque chose bloque

Symptôme	Cause probable	Que faire
`Not found: Dataset ... was not found in location US`	Votre jeu de données `shopflow` n'est pas en EU	Recréez-le avec `OPTIONS (location = 'EU')`
`Access Denied` sur le jeu de l'enseignant	Le partage n'est pas encore actif pour votre compte	Prévenez l'enseignant
`DML queries are not allowed in the free tier`	Vous avez utilisé `INSERT` ou `UPDATE` dans le sandbox	Réécrivez avec `CREATE OR REPLACE TABLE ... AS SELECT`
0 octet traité sur une requête relancée	Le cache est actif	Décochez « Utiliser les résultats mis en cache »
`ModuleNotFoundError: shopflow_gen` (Databricks)	Mauvais chemin dans `sys.path.append`	Copiez le chemin exact du dossier `tp1` depuis le navigateur *Workspace*
Pas de profil de requête disponible (Databricks)	Résultat servi par le cache	`SET use_cached_result = false;` puis relancez